



SE618 Logistic Case Study Analysis
Academic Year 2025–2026, Semester 2

SERGE

An AI Agent for Cross-System
Supply Chain Diagnostics via MCP

Final Report

Design, Implementation, and Empirical Evaluation

Authors

Gauthier Schmitt	6822807466
Louis Grignola	6822807516
Matteo Schwarz	6822807557
Maël Tellus	6822807532

Supervisor

Ajarn Warut

School of Manufacturing Systems and Mechanical Engineering
Sirindhorn International Institute of Technology — Thammasat University

April 2026

Contents

1	Introduction	1
1.1	The Problem	1
1.2	The Question	1
1.3	SERGE	1
1.4	Contributions	1
1.5	Headline Results	1
1.6	Report Map	2
2	Background	3
2.1	Large Language Models	3
2.2	AI Agents	3
2.3	Model Context Protocol	3
3	Related Work	4
3.1	AI Agents for Supply Chain	4
3.2	LLM-as-Judge	4
3.3	Gaps Addressed	4
4	Business and Supply Chain Modelisation	5
4.1	The Business: Siam Mango Co.	5
4.2	Four Bounded Contexts	5
4.3	Entity Model	5
4.4	Dataset	6
5	Architecture	8
5.1	Agent Layer	8
5.2	MCP Layer	8
5.3	End-to-End Example	9
6	Evaluation Methodology	10
6.1	Three-Stage Pipeline	10
6.2	Judge Rubric: Flexible on Form, Strict on Facts	10
6.3	Complexity Levels	10
6.4	Scoring and Metrics	10
6.5	Setup	11

7	Scenarios	12
7.1	Structure	12
7.2	The 27 Scenarios	12
7.3	One Scenario per Level	12
7.4	Ground-Truth Construction	13
8	Results	14
8.1	Headline Metrics	14
8.2	Refinement Cycle	14
8.3	By Complexity Level	15
8.4	By Use Case	16
8.5	Failure Taxonomy	16
8.6	Resolution Time	17
8.7	Cross-System Coverage	17
8.8	Qualitative Behaviour	18
9	Discussion	19
9.1	What Worked	19
9.2	Limitations	19
9.3	Threats to Validity	19
9.4	Implications for Practice	19
10	Conclusion	20
	References	21
A	MCP Tool Reference	24
B	Scenario Metadata	25
	Glossary	26
	Acronyms	27

List of Figures

4.1	Conceptual class diagram. Four bounded contexts; solid arrows are associations within a context, dashed arrows are cross-context references by identifier. Field-level detail is omitted for clarity.	6
5.1	The three-layer stack. The user speaks natural language to their desktop client; the client runs SERGE as an MCP subprocess; the server exposes 35 read-only tools grouped by the four systems.	8
8.1	Headline metrics. Run 1 (grey) vs Run 2 (blue). Every dimension improves; hallucination rate drops to zero.	14
8.2	Weighted score by complexity level.	15
8.3	Pass rate by business use case.	16
8.4	Failure taxonomy for both passes.	16
8.5	Measured agent resolution time per complexity level (Run 2).	17
8.6	Average systems touched per run, by level (Run 2).	17

List of Tables

4.1	Supply chain systems and their entities.	5
5.1	35 MCP tools distributed by system. Full reference in Appendix A.	8
6.1	Five complexity levels. Complexity is orthogonal to the business use case; each scenario is tagged with both.	10
6.2	Judge scoring dimensions.	11
7.1	The 27 evaluation scenarios. Full metadata in Appendix B.	12
B.1	Scenario metadata.	25

1. Introduction

1.1 The Problem

Supply chains run on four enterprise systems: [Supplier Relationship Management \(SRM\)](#) (procurement), [Warehouse Management System \(WMS\)](#) (inventory), [Order Management System \(OMS\)](#) (sales), [Transport Management System \(TMS\)](#) (shipping). These systems come from different vendors and do not talk to each other. A GEODIS survey reports only 6% of companies achieve full visibility across them [15]; operations managers spend 15 to 30 minutes per investigation manually stitching data from three or four interfaces [16, 17].

Most cross-system questions are blocked by this fragmentation: *why is this order late?, which customers are at risk if product X runs out?, is the chosen carrier appropriate for that product?*

1.2 The Question

Can an AI agent with read-only access to the four systems answer realistic operational questions at human-level factual accuracy, while significantly reducing investigation time?

1.3 SERGE

We build **SERGE**, a read-only agent that sits over the four supply chain systems via the Model Context Protocol [13]. A simulated B2B mango distributor is used as the target domain; its four systems are exposed as 35 read-only [Model Context Protocol \(MCP\)](#) tools. Claude Code acts as the reasoning agent, invoked from the user’s desktop client.

The system is evaluated on 27 scenarios spanning five complexity levels (L1 trivial to L5 expert). Each scenario is run three times, judged by a separate [Large Language Model \(LLM\)](#) on four dimensions (factual accuracy, completeness, hallucination, reasoning), and aggregated. Two passes are reported: a baseline and a refined run after one iteration cycle.

1.4 Contributions

1. A plug-and-play [MCP](#) architecture for supply chain systems, agent-agnostic by design.
2. A read-only diagnostic agent pattern, complementary to the optimisation-centric supply chain [Artificial Intelligence \(AI\)](#) literature.
3. An [LLM](#)-as-judge methodology with an explicit *flexible-on-form, strict-on-facts* principle and a structured failure taxonomy.
4. A reusable benchmark of 27 scenarios at five complexity levels.

1.5 Headline Results

Refined pass (Run 2): 94% pass rate over 81 evaluations, weighted score 4.84/5.0, **zero hallucinations**, 100% pass on L4 (complex cross-system) scenarios. Resolution time averages about 13 s on L1 and 72 s on L5.

1.6 Report Map

Chapter 2: background on LLMs, agents, and MCP. Chapter 3: related work. Chapter 4: the modelled business, the entity model, and how the dataset was seeded. Chapter 5: the agent–MCP–backend stack. Chapter 6: evaluation methodology. Chapter 7: the 27 scenarios. Chapter 8: empirical results, including the refinement cycle. Chapter 9: discussion. Chapter 10: conclusion.

2. Background

2.1 Large Language Models

A **LLM** is a deep neural network trained on very large text corpora. All modern **LLMs** are built on the **Transformer** architecture [1]. Loss scales predictably with model size, data size, and compute [2, 3]; capabilities such as multi-step reasoning and tool use *emerge* past capability thresholds rather than improving linearly [4]. In April 2026 the practical models are Claude 4.5, GPT-5, Llama 4, and Mistral Large 3. This work uses Claude Sonnet through the Claude Code client.

A **LLM** on its own is a text-to-text function: no memory, no tools, no ability to act. For supply chain diagnostics—which require reading state from multiple sources—the **LLM** must be embedded in a larger system that provides tools and a reasoning loop. That system is the agent.

2.2 AI Agents

An **agent** wraps an **LLM** with (i) a set of tools it can call, (ii) a mechanism to observe the tool result, and (iii) a loop that decides what to call next and when to stop [7]. The seminal formulation is **ReAct** [5]: the **LLM** alternates reasoning steps and tool calls until it emits a terminal response.

The agent used here, Claude Code [10], goes beyond a plain **ReAct** loop: extended thinking is built in, sub-agent spawning is available, reusable workflows can be loaded on demand, and context is actively managed [11]. Tool description quality is the dominant factor in tool-call reliability [12, 6, 8], a point we return to in the architecture chapter.

2.3 Model Context Protocol

Without a standard integration layer, every agent-system pair needs bespoke code; with N agents and M systems the integration matrix has $N \times M$ cells. The **MCP** [13, 14], introduced by Anthropic in November 2024, collapses this matrix to $N + M$: each agent and each system implements **MCP** once and all combinations work. Primitives are *tools* (callable functions with typed parameters), *resources* (readable data), and *prompts* (templates); transports are **stdio** (subprocess) or HTTP. SERGE uses tools only, over **stdio**.

MCP is the right glue for SERGE for three reasons: it is agent-agnostic (swap Claude for any **MCP** client without touching the backend), typed and self-describing (new agents connect without extra configuration), and trivial to deploy locally (no network services to run).

3. Related Work

3.1 AI Agents for Supply Chain

Multi-agent LLM systems have been shown to reduce the bullwhip effect in simulation [18]. Generative planning has been deployed at industrial scale by JD.com (SCPA) [19]. Knowledge-graph query agents have been applied to disruption monitoring [20], and multi-agent retrieval pipelines have been used to ingest unstructured supply chain communications [21]. All four systems aim at autonomous action. SERGE takes the complementary read-only diagnostic position: an agent that reads current state and explains it, before anything is written back.

A systematic review of 98 peer-reviewed studies [22] observes that about 45% of published AI-for-supply-chain work does not fully specify its architecture. The field is still methodologically young.

3.2 LLM-as-Judge

Zheng et al. [23] introduced the LLM-as-judge framework with MT-Bench and Chatbot Arena, reporting over 80% agreement with human raters. Follow-up surveys [24, 25] catalogue common biases (position, verbosity, self-enhancement) and their mitigations. Shankar et al. [26] examine rubric design and alignment with human judgement. Our methodology (Chapter 6) uses pointwise judging with a domain-specific rubric and adds an explicit *flexible-on-form* rule after observing the strictness failure mode in our own baseline run.

3.3 Gaps Addressed

Four gaps identified in our Week 3 state-of-the-art review motivate this work: (1) no standardised supply chain agent benchmark, (2) no plug-and-play architecture connecting supply chain systems to agents, (3) read-only diagnostic agents underexplored, (4) no supply-chain-specific LLM-as-judge rubric. SERGE addresses gaps 2–4 directly; the 27 scenarios released with this work contribute to gap 1.

4. Business and Supply Chain Modelisation

4.1 The Business: Siam Mango Co.

Siam Mango Co. is a fictional small B2B mango distributor in Bangkok. It sources from 5 Thai farms, stores in 1 warehouse (Bang Na), sells 5 fresh and 3 processed products to 12 business customers (hotels, supermarkets, food halls), and delivers through 3 carriers (1 ground, 2 refrigerated). The domain is small enough to audit manually and rich enough to exercise four realistic constraints: perishability (fresh shelf life 3–10 days), cold chain (refrigerated products require refrigerated carriers), supplier reliability (0–100% on-time in the data), and multi-sourcing (most products available from two farms at different costs).

4.2 Four Bounded Contexts

The four supply chain systems map to four bounded contexts in the Domain-Driven Design sense: each has its own ubiquitous language and its own state; cross-context references are carried by stable identifiers.

System	Domain	Entities
SRM	Procurement	Supplier, SupplierProduct, SupplierOrder
WMS	Inventory	Product, Warehouse, InventoryRecord, StockMovement
OMS	Sales	Customer, CustomerOrder
TMS	Shipping	Carrier, Shipment, TrackingEvent

Table 4.1: Supply chain systems and their entities.

4.3 Entity Model

Figure 4.1 shows the conceptual class diagram. Solid arrows are associations within a context; dashed arrows are cross-context references carried by identifier. One design point is worth calling out: prices are not duplicated on order lines. A **SupplierProduct** bridge entity carries the purchase cost; sale prices live on the **Product** entity. Order lines reference a product-level entity and a quantity—nothing else. This removes a whole class of “price on line vs price in catalogue” inconsistencies and sharpens the domain language the agent reads in its system prompt.

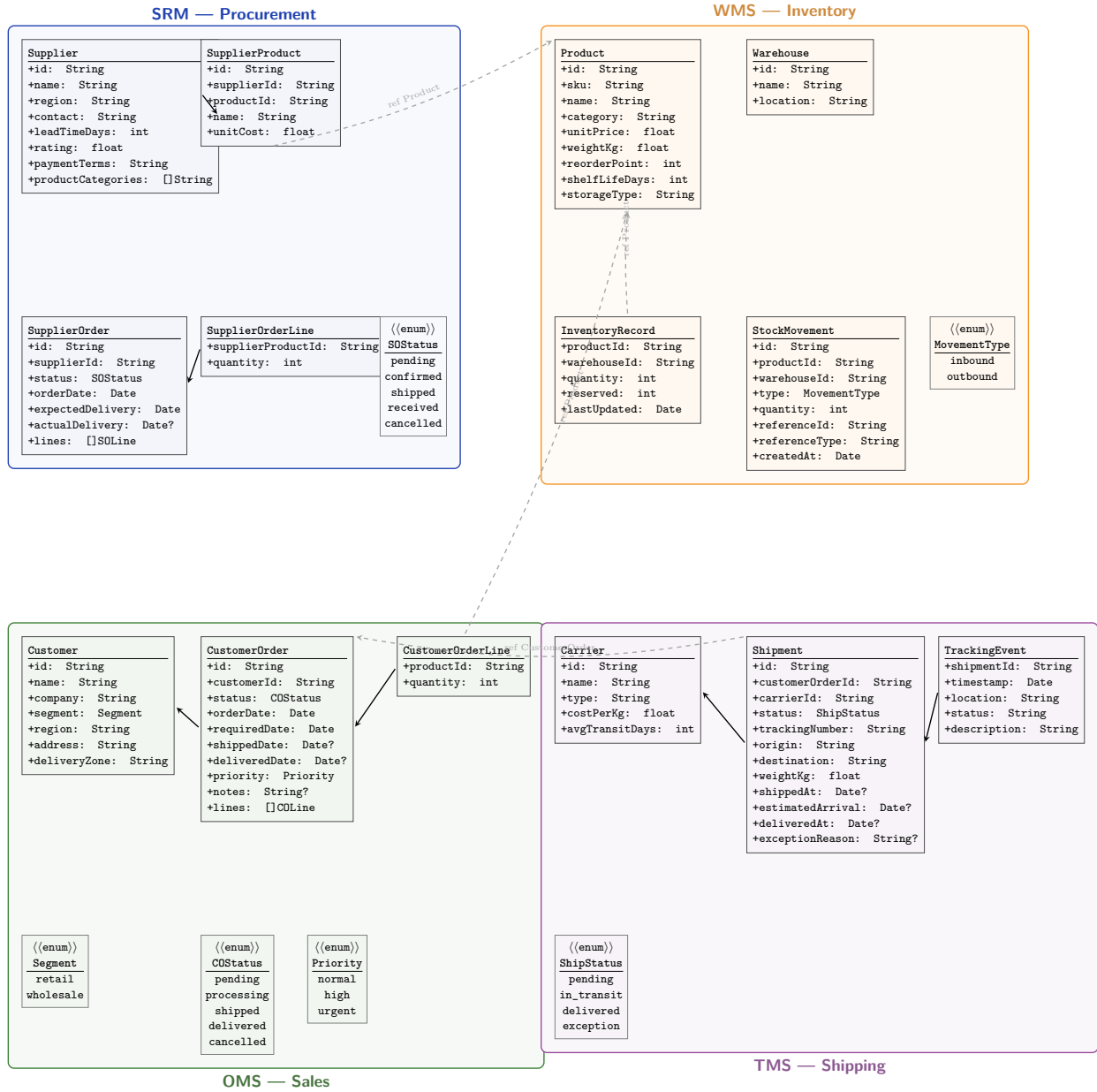


Figure 4.1: Conceptual class diagram. Four bounded contexts; solid arrows are associations within a context, dashed arrows are cross-context references by identifier. Field-level detail is omitted for clarity.

Each transactional entity moves through a status lifecycle. A **SupplierOrder** goes pending → confirmed → shipped → received. A **CustomerOrder** goes pending → processing → shipped → delivered; an order that cannot be fulfilled (insufficient stock) stays in processing, which is the state of the flagship L4 scenario. A **Shipment** goes pending → in_transit → delivered, with exception as a branch for delivery problems. cancelled is a terminal state on each lifecycle.

4.4 Dataset

The dataset combines 30 hand-authored scenario orders, 18 scenario **Purchase Order (PO)**s, 23 scenario shipments (identifiers below 100) with 470 additional orders, 42 **PO**s, and 470 shipments generated on top (identifiers from 100 onward). Volumes: 500 customer orders, 60

supplier orders, 493 shipments, 1,431 tracking events, 965 stock movements.

Seeding. The dataset was bootstrapped by an agent producing realistic initial entity definitions, then expanded deterministically by a reproducible Go seeding script (fixed random seed, five months of history). An agent review pass verified internal consistency of the generated data. The evaluation ground truth was built with the same agent-assisted approach: for each scenario, the expected answer was cross-checked against the actual dataset through exploratory queries, ensuring the ground truth reflects what exists in the data rather than what we thought existed at scenario-design time. The identifier convention (scenario below 100, generated from 100) means the dataset can be regenerated without disturbing ground truth.

Embedded patterns. The 30 hand-authored scenarios encode specific operational situations used as ground truth during evaluation: a stock-out cascade on **PROD-001**, a supplier with a consistent delay pattern (**SUP-002**), a ground carrier assigned to a refrigerated product in error (**SHP-009**), a split shipment across two carriers (**ORD-023**), and a single order (**ORD-007**, 50 units) larger than the product’s reorder point.

5. Architecture

SERGE is a three-layer stack: a natural-language agent in the user’s desktop client, an [MCP](#) server exposing the supply chain domain, and the four data contexts (SRM, WMS, OMS, TMS) behind the server.

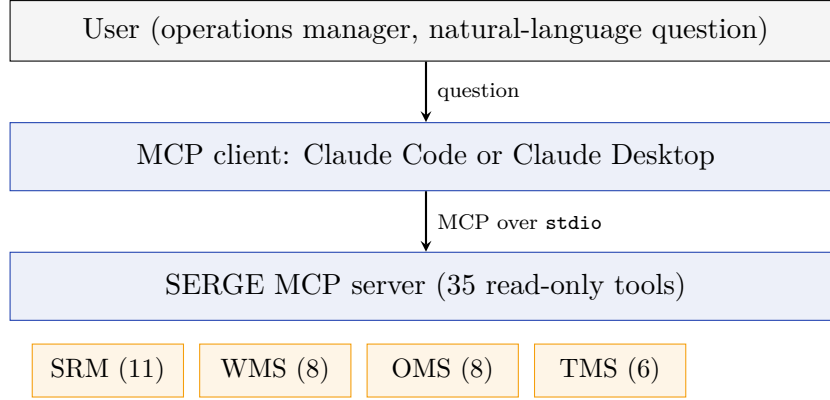


Figure 5.1: The three-layer stack. The user speaks natural language to their desktop client; the client runs SERGE as an MCP subprocess; the server exposes 35 read-only tools grouped by the four systems.

5.1 Agent Layer

Any [MCP](#)-compatible client can be used; we run Claude Code. The agent receives the user’s question together with a project-level system prompt (`CLAUDE.md`) that describes the business, enumerates every entity and status, states the critical business rules (refrigerated products require refrigerated carriers; overdue = past required date and not delivered or cancelled), and lists common cross-system reasoning patterns. A final block of *Rules to Avoid Mistakes* (added in the refinement cycle of Chapter 8) instructs the agent to never invent identifiers, to distinguish active from historical data, and to list all matching items rather than a subset.

5.2 MCP Layer

The 35 tools are grouped by system and distributed as shown in Table 5.1. All tools are read-only *at the protocol level*: no mutation tool is registered, so the agent cannot write to the backend even if prompted to. Tool descriptions follow Anthropic’s guidance [12]: each states what the tool returns, what its parameters expect, and—for critical tools—an example value. Description quality was re-reviewed between the two evaluation passes; small tightenings on ambiguous tools measurably improved tool-selection accuracy.

System	Tools	Examples
SRM	11	<code>get_supplier</code> , <code>compare_suppliers</code> , <code>list_supplier_orders_by_status</code>
WMS	8	<code>check_inventory</code> , <code>list_low_stock</code> , <code>get_stock_movements</code>
OMS	8	<code>get_customer_order</code> , <code>search_customer_orders_by_product</code>
TMS	6	<code>get_shipment</code> , <code>track_customer_order</code> , <code>list_exception_shipments</code>

Table 5.1: 35 MCP tools distributed by system. Full reference in Appendix A.

5.3 End-to-End Example

For the question *Why has order ORD-007 not shipped yet?*, a typical run is four tool calls: `get_customer_order` (order is processing), `track_customer_order` (no shipment exists), `check_inventory` (8 units on hand against 50 required), `list_supplier_orders_by_status` (incoming PO arrives after the required date). The agent then synthesises the causal chain. No component in the backend is aware the question is diagnostic; the intelligence lives entirely in the agent, which is why swapping MCP clients requires no backend change.

6. Evaluation Methodology

6.1 Three-Stage Pipeline

Each scenario is run three times through a three-stage pipeline. The **agent run** sends the question to Claude Code with the 35 [MCP](#) tools enabled and captures the response, the tool trace, the duration, and the cost. The **judge run** sends the agent response plus the scenario’s ground truth to a separate [LLM](#) (also Claude Sonnet) with a structured rubric, which returns per-dimension scores, a pass flag, and a failure category. The **aggregation** step (Python) produces per-scenario, per-level, per-use-case, and failure-taxonomy statistics. The pipeline is resumable: transient failures are retried with exponential back-off, and quota exhaustion is detected explicitly so a run can be continued in a later session.

6.2 Judge Rubric: Flexible on Form, Strict on Facts

Naive [LLM](#)-as-judge setups over-penalise presentation. In our baseline run, one scenario scored 4.18/5.0 on the weighted dimensions but was marked failing because one item of the critical list was not literally mentioned, despite the agent’s conclusion being identical to the expected one. The refined judge prompt is organised around one principle: *flexible on form, strict on facts*. Wording variations, alternative tool paths, extra analysis, and answers that extend the ground truth with legitimate data are accepted; wrong values, wrong identifiers, and genuine fabrication are penalised. Identifiers that match the known patterns (ORD-XXX, P0-XXX, ...) are not flagged as hallucinations unless they contradict the ground truth.

6.3 Complexity Levels

Level	Name	N	Tools	Tests
L1	Trivial	4	1	Single entity lookup
L2	Simple	6	1–2	One-system filter or aggregation
L3	Moderate	6	2–3	Cross-reference between two systems
L4	Complex	6	3–5	Causal reasoning across 3–4 systems
L5	Expert	5	6+	Full diagnostic plus actionable recommendation

Table 6.1: Five complexity levels. Complexity is orthogonal to the business use case; each scenario is tagged with both.

6.4 Scoring and Metrics

The judge rates each response on four 1-to-5 dimensions with weights in [Table 6.2](#). A run **passes** if the weighted score is ≥ 3.5 *and* all critical facts are present *and* no hallucination is detected.

Dimension	Weight	Rewards
Factual accuracy	40%	All values, IDs, dates, statuses match.
Completeness	30%	Critical facts present.
No hallucination	20%	No fabricated content.
Reasoning	10%	Internal coherence.

Table 6.2: Judge scoring dimensions.

The aggregator also computes the *pass rate*, *functional consistency* (conclusion matches ground truth across repeated runs), the *hallucination rate*, *system coverage* (expected systems actually queried), *tool efficiency* (expected min / actual tool count), and the *measured agent resolution time* per run. Each judge output records a *failure category* from `{none, missed_fact, wrong_value, hallucination, bad_reasoning, refused}`, which powers the failure analysis in Chapter 8.

6.5 Setup

Each pass is $27 \text{ scenarios} \times 3 \text{ runs} = 81 \text{ evaluations}$. Both agent and judge run on Claude Sonnet. Two passes are reported: **Run 1** (baseline, 17 April 2026) and **Run 2** (refined, 18 April 2026, after the iteration cycle documented in §8.2).

7. Scenarios

7.1 Structure

Each scenario is a JSON document with a natural-language question and a tiered ground truth: **critical** facts must all appear for the run to pass; **important** facts are expected but not pass-critical; **must-not-contain** facts are hard negatives the answer must avoid. Metadata fields (expected systems, minimum tool count, estimated human minutes) drive the aggregator’s coverage, efficiency, and time-savings metrics.

7.2 The 27 Scenarios

The scenarios span five business use cases: UC-1 order status, UC-2 root cause, UC-3 supplier/sourcing, UC-4 operational monitoring, UC-5 planning. Use case and complexity level are orthogonal.

ID	Level	UC	Title
L1-01	L1	UC-4	Product Lookup
L1-02	L1	UC-4	Customer Lookup
L1-03	L1	UC-3	Supplier Profile
L1-04	L1	UC-4	List Carriers
L2-01	L2	UC-1	Basic Order Status
L2-02	L2	UC-4	Customer Order History
L2-03	L2	UC-4	Overdue Orders
L2-04	L2	UC-4	Low Stock Alert
L2-05	L2	UC-3	Supplier Order History
L2-06	L2	UC-4	Shipment Exceptions
L3-01	L3	UC-1	Order Tracking with Current Location
L3-02	L3	UC-4	Product Demand View
L3-03	L3	UC-3	Supplier Side-by-Side Comparison
L3-04	L3	UC-3	Sourcing Options
L3-05	L3	UC-1	Late Delivery Check
L3-06	L3	UC-4	Product With Current Inventory
L4-01	L4	UC-2	Order Delay Diagnosis
L4-02	L4	UC-2	Carrier Mismatch Diagnosis
L4-03	L4	UC-3	Supplier Reliability Pattern
L4-04	L4	UC-4	Operational Scan
L4-05	L4	UC-2	Customer Impact Assessment
L4-06	L4	UC-1	Split Shipment Status
L5-01	L5	UC-4	Full Product Supply Chain History
L5-02	L5	UC-4	Full Operational Health Check
L5-03	L5	UC-5	Fulfillment Feasibility Plan
L5-04	L5	UC-3	Sourcing Strategy Review
L5-05	L5	UC-5	Reorder Point Analysis

Table 7.1: The 27 evaluation scenarios. Full metadata in Appendix B.

7.3 One Scenario per Level

L1-01 Product Lookup. “*Tell me about PROD-001.*” One **WMS** call; the agent must pick the right tool on the first try.

L2-04 Low Stock Alert. “*What products are below their reorder point?*” One **WMS** call; the agent must return all three low-stock products, not a subset.

L3-01 Order Tracking. “*Where is ORD-005 right now?*” **OMS** then **TMS**; the agent

synthesises carrier, current location, and ETA from two systems.

L4-01 Order Delay Diagnosis. “*Why has ORD-007 not shipped?*” The canonical cross-system diagnostic: **OMS** (processing) → **TMS** (no shipment) → **WMS** (stock shortage) → **SRM** (incoming **PO** arrives too late).

L5-02 Operational Health Check. “*Full health check of the supply chain.*” Scan all four systems, surface overdue orders, exceptions, low stock, supplier reliability concerns; produce at least one cross-system insight.

7.4 Ground-Truth Construction

Ground truths are derived from the scenario data (identifiers below 100). The baseline set was written against scenario data only, which caused the judge to flag legitimate extensions to generated data as hallucinations. The refinement cycle (Chapter 8) either rewrote the ground truths to focus on the *shape* of the expected answer rather than exhaustive ID lists, or tightened the judge prompt to accept legitimate extensions.

8. Results

Two evaluation passes were run: a baseline (**Run 1**) and a refined pass (**Run 2**) after one iteration cycle. Run 1 completed 80 of 81 evaluations (one L5 run hit the initial \$0.50 per-run ceiling); Run 2, with the ceiling raised to \$1.50, completed all 81.

8.1 Headline Metrics

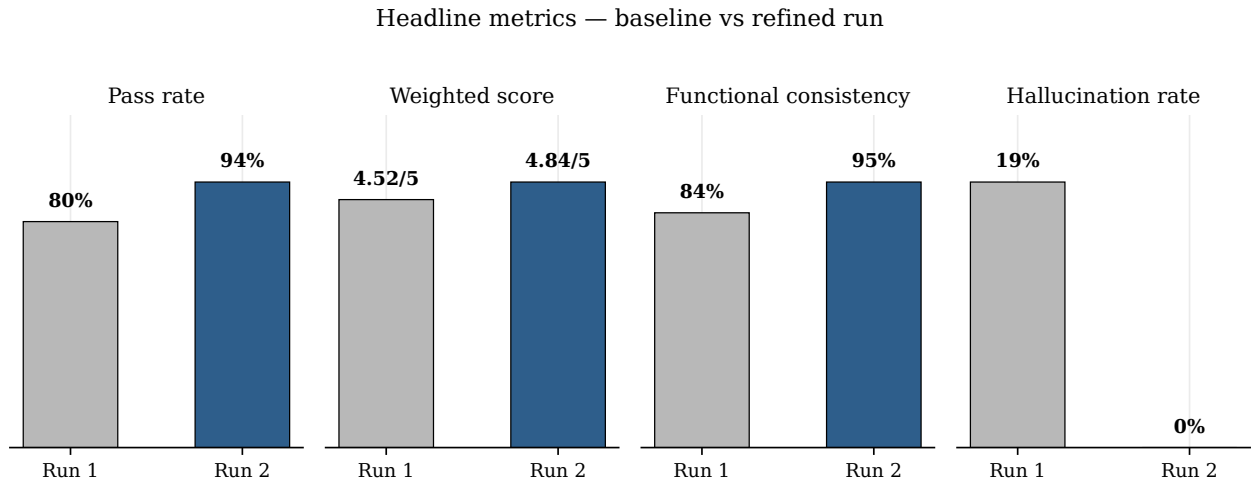


Figure 8.1: Headline metrics. Run 1 (grey) vs Run 2 (blue). Every dimension improves; hallucination rate drops to zero.

Pass rate: 80% → 94%. Weighted score: 4.52 → 4.84 out of 5.0. Functional consistency: 84% → 95%. Hallucination rate: 19% → **0%**. The refined agent fabricates nothing across 81 runs.

Result

Across 81 refined evaluations: **94% pass rate**, **4.84/5.0** weighted score, **zero fabricated data**.

8.2 Refinement Cycle

Three changes between the two passes account for the improvement.

Ground-truth recalibration. Five scenarios in Run 1 passed at 0% despite high underlying scores. Inspection showed the ground truths listed scenario-only identifiers while the agent correctly used the full dataset, and the judge flagged the extensions as hallucinations. Four were rewritten to focus on answer shape; one had a single over-strict critical fact demoted to important.

Judge prompt revision. An explicit rule was added: identifiers matching the domain patterns are not hallucinations unless they contradict the ground truth. The **hallucination** failure category is now reserved for genuine fabrication (names outside the domain, impossible values, contradictions).

Agent prompt revision. A *Rules to Avoid Mistakes* section was added to the agent’s system prompt: never invent identifiers, distinguish active from historical data, list all matching

items, compute metrics over the full result set, respect the critical business rules on cold chain and reorder points.

8.3 By Complexity Level

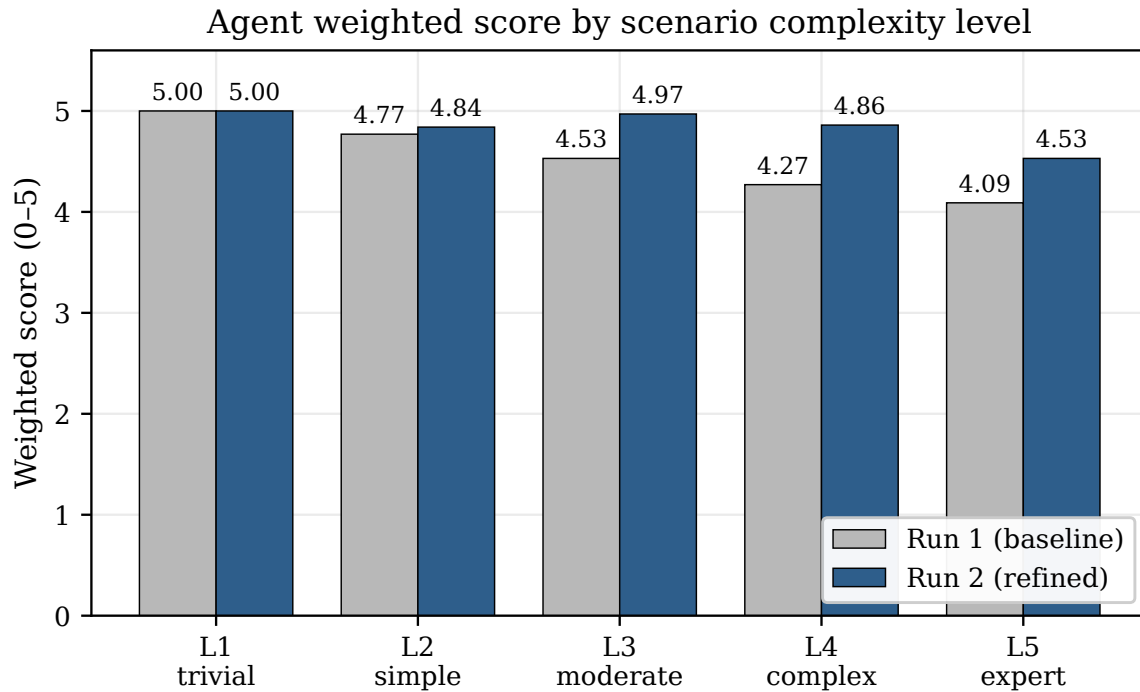


Figure 8.2: Weighted score by complexity level.

Accuracy decreases with complexity, as expected, but flattens in Run 2. L1 stays at 5.0. L3 rises $4.53 \rightarrow 4.97$. **L4 rises $4.27 \rightarrow 4.86$ and passes at 100%**—the canonical cross-system diagnostics all succeed. L5 rises $4.09 \rightarrow 4.53$ and remains the hardest level.

8.4 By Use Case

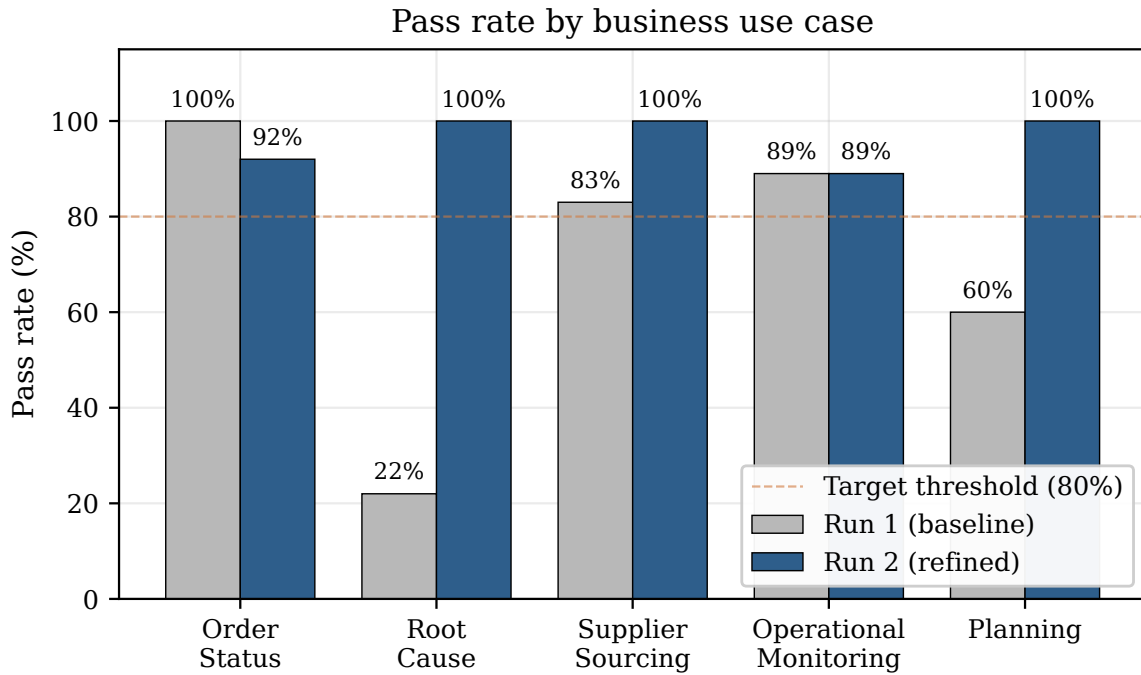


Figure 8.3: Pass rate by business use case.

UC-2 (Root Cause) rises from 22% to 100%—the largest shift. UC-3 and UC-5 also reach 100%. UC-4 stays at 89%. UC-1 regresses marginally (100% $\rightarrow$ 92%) because of one partial L2 answer, which does not affect the aggregate.

8.5 Failure Taxonomy

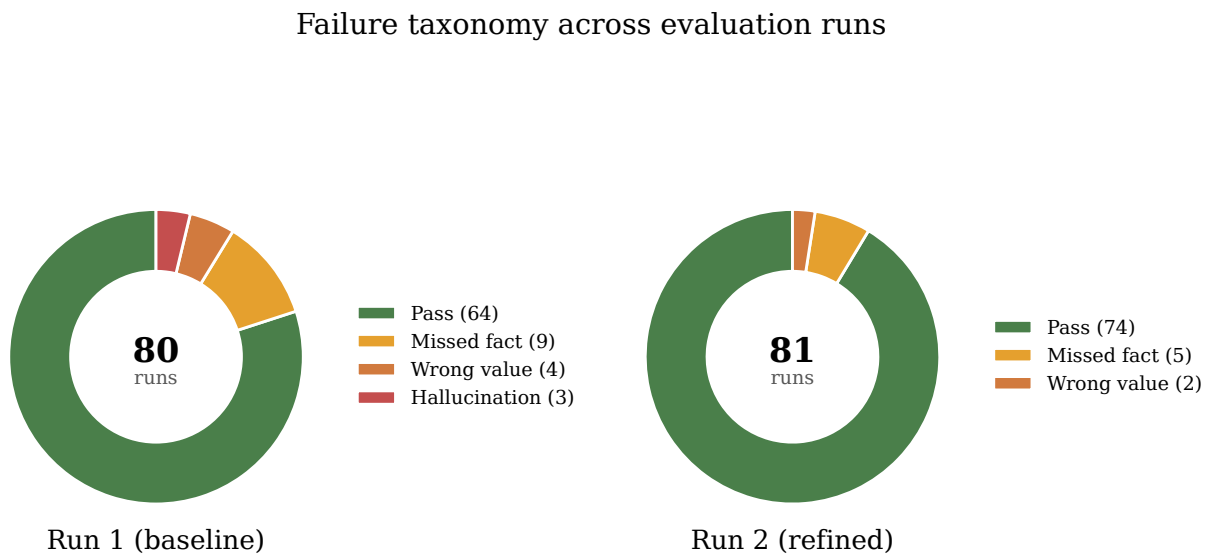


Figure 8.4: Failure taxonomy for both passes.

hallucination falls from 4% to 0%, wrong_value from 5% to 2%, missed_fact from 11% to 6%. The zero-hallucination outcome is the combined effect of the agent rule (never invent)

and the judge rule (no false positive on legitimate extensions).

8.6 Resolution Time

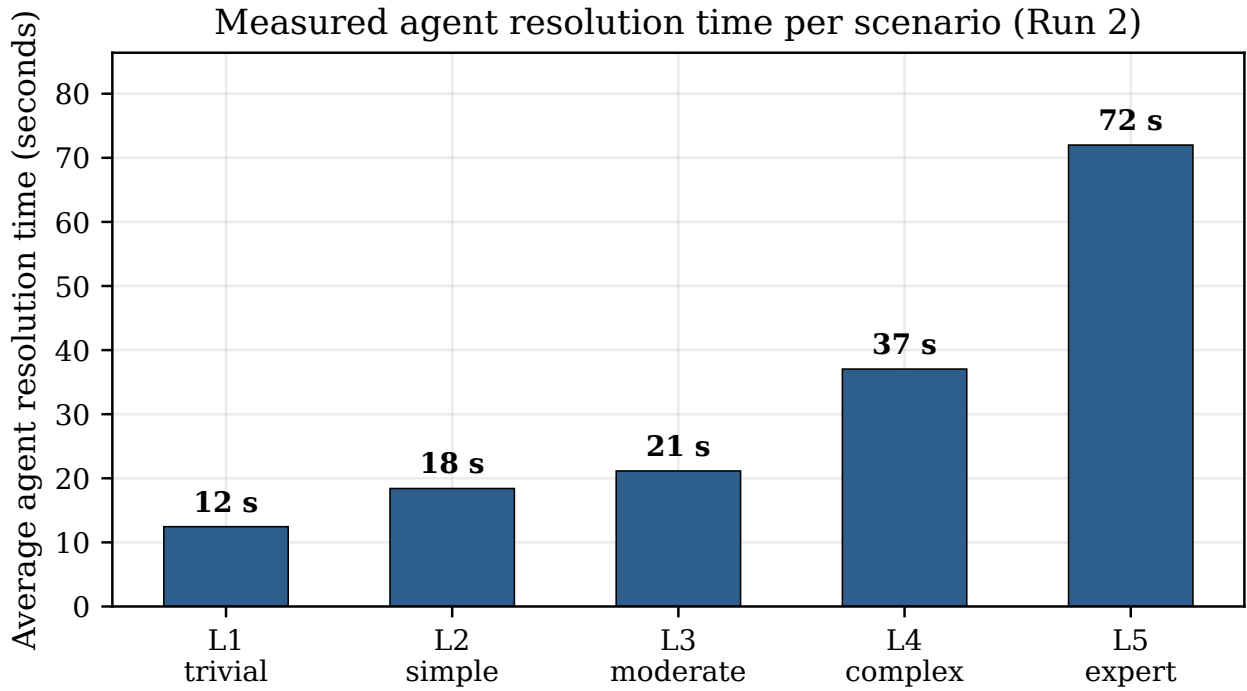


Figure 8.5: Measured agent resolution time per complexity level (Run 2).

Measured averages per scenario: L1 $\approx$ 13 s, L2 $\approx$ 18 s, L3 $\approx$ 21 s, L4 $\approx$ 37 s, L5 $\approx$ 72 s. Across the 81 Run 2 evaluations the agent spent 43.5 min in total, averaging about 32 s per run.

8.7 Cross-System Coverage

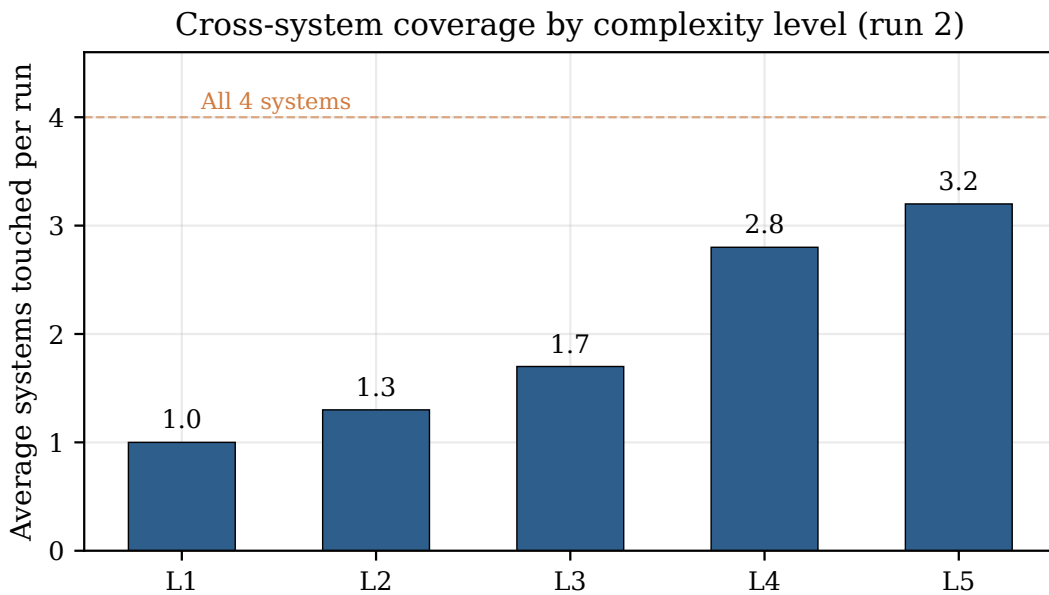


Figure 8.6: Average systems touched per run, by level (Run 2).

L1 averages 1 system; L4 averages 2.8; L5 averages 3.2. The agent does chain across systems when the question requires it, and stays scoped when it does not.

8.8 Qualitative Behaviour

Three Run 2 responses illustrate behaviour that scalar metrics do not fully capture. On the carrier-mismatch diagnosis, the agent identifies the storage-type mismatch, references the SHP-009 refrigeration failure as evidence, and lists refrigerated alternatives—a production-quality incident report. On the full product history, the agent chains six tool calls, computes a running stock balance from movements, produces an ASCII flow diagram, and ends with a two-sentence takeaway. On the fulfilment-feasibility question, it computes the 4-unit gap, compares the two refrigerated carriers, and proposes three options with trade-offs.

9. Discussion

9.1 What Worked

MCP delivered on its agent-agnostic promise. The same backend runs unchanged under Claude Code and Claude Desktop; any MCP-compatible client would work without modification.

Flexible-on-form judging lifted the pass rate by 14 points. The most impactful methodological change between the two runs was the judge rubric, not the agent. The agent was already correct in most scenarios that failed Run 1.

Tool descriptions and system prompt are the high-leverage artefacts. The iteration budget was spent on these two surfaces, not on code. The measured accuracy improvement between Run 1 and Run 2 came almost entirely from prompt-level changes.

9.2 Limitations

The dataset is simulated and contains patterns we designed; a production dataset would include noise and edge cases not exercised here. The domain is narrow (one small distributor in Bangkok), so generalisation across verticals is untested. Both agent and judge are Claude Sonnet, so cross-family comparison is left to future work. The judge is itself an LLM, and is not cross-validated against humans.

9.3 Threats to Validity

A critic could argue that the Run 1 $\rightarrow$ Run 2 improvement is partly due to ground-truth softening. Three observations push back. First, the hallucination rate—a purely factual metric independent of rubric tolerance—fell from 19% to 0%. Second, functional consistency rose from 84% to 95% and measures whether the conclusion is correct, not whether a list of facts is matched. Third, L1 and L2 scenarios, none of which had their ground truths modified, also improved marginally between runs. The ground-truth recalibration explains part of the L3–L4 improvement but not the hallucination and consistency gains.

We report *measured* agent resolution times (13–72 s per scenario, averaging 32 s per run) but do not claim a percentage saving over a human. Comparing against a human baseline would require measured data from real operators, which we do not have.

9.4 Implications for Practice

Three practical implications. **Read-only is a defensible initial scope:** the diagnostic value alone justifies the deployment, and write access can be added later once trust is established. **MCP integration is cheap insurance:** the investment is small and preserves optionality on the agent side. **Tool descriptions and the system prompt deserve explicit ownership,** as much as the data model itself; both dominate the measured agent accuracy.

10. Conclusion

SERGE is a read-only AI agent connecting four supply chain systems through the MCP. Over a simulated B2B mango distributor in Bangkok, and evaluated on 27 scenarios spanning five complexity levels, the refined pass reaches a 94% pass rate, a 4.84/5.0 weighted score, zero hallucinations across 81 evaluations, and 95% functional consistency. Measured agent resolution time averages 13s on trivial scenarios and 72s on expert-level ones.

Four contributions follow. A plug-and-play MCP architecture for supply chain systems, agent-agnostic by design. A read-only diagnostic agent pattern, complementary to the optimisation-centric literature. A flexible LLM-as-judge methodology with a structured failure taxonomy, validated through an explicit iteration cycle. A reusable benchmark of 27 scenarios released with the work.

Three directions extend this naturally: a predictive layer for demand and disruption forecasting; controlled write access behind a human-in-the-loop approval workflow; and a cross-agent comparison holding the MCP backend and the scenarios fixed. We leave these to future work.

References

Bibliography

- [1] A. Vaswani, N. Shazeer, N. Parmar, J. Uszkoreit, L. Jones, A. N. Gomez, Ł. Kaiser, and I. Polosukhin. Attention is all you need. *Advances in Neural Information Processing Systems*, 30:5998–6008, 2017.
- [2] J. Kaplan, S. McCandlish, T. Henighan, T. B. Brown, B. Chess, R. Child, S. Gray, A. Radford, J. Wu, and D. Amodei. Scaling laws for neural language models. *arXiv:2001.08361*, 2020.
- [3] J. Hoffmann, S. Borgeaud, A. Mensch, et al. Training compute-optimal large language models. *arXiv:2203.15556*, 2022.
- [4] J. Wei, Y. Tay, R. Bommasani, et al. Emergent abilities of large language models. *Transactions on Machine Learning Research*, 2022.
- [5] S. Yao, J. Zhao, D. Yu, N. Du, I. Shafran, K. Narasimhan, and Y. Cao. ReAct: Synergizing reasoning and acting in language models. In *Proceedings of ICLR*, 2023.
- [6] T. Schick, J. Dwivedi-Yu, R. Dessì, R. Raileanu, M. Lomeli, L. Zettlemoyer, N. Cancedda, and T. Scialom. Toolformer: Language models can teach themselves to use tools. In *Proceedings of NeurIPS*, 2023.
- [7] L. Weng. LLM powered autonomous agents. <https://lilianweng.github.io/posts/2023-06-23-agent/>, 2023.
- [8] S. G. Patil, T. Zhang, X. Wang, and J. E. Gonzalez. Gorilla: Large language model connected with massive APIs. In *Proceedings of NeurIPS*, 2024.
- [9] Anthropic. Building effective agents. <https://www.anthropic.com/research/building-effective-agents>, 2024.
- [10] Anthropic. How Claude Code works. <https://code.claude.com/docs/en/how-claude-code-works>, 2025.
- [11] Anthropic. Effective context engineering for AI agents. <https://www.anthropic.com/engineering/effective-context-engineering-for-ai-agents>, 2025.
- [12] Anthropic. Writing tools for agents. <https://www.anthropic.com/engineering/writing-tools-for-agents>, 2025.
- [13] Anthropic. Model context protocol. <https://www.anthropic.com/news/model-context-protocol>, 2024.
- [14] Model context protocol specification (2025-11-25). <https://modelcontextprotocol.io/specification/2025-11-25>, 2025.
- [15] GEODIS. Supply chain visibility survey. Cited by Descartes Systems, 2024.
- [16] Synchronizing ERP, WMS, and TMS systems for supply chain optimization. *Supply & Demand Chain Executive / AWS*, 2024.

- [17] Omniful. Unified dashboard: Connecting ERP, OMS, WMS, and TMS, 2026.
- [18] V. Jannelli, S. Schoepf, M. Bickel, T. Netland, and A. Brintrup. Agentic LLMs in the supply chain: Towards autonomous multi-agent consensus-seeking. *International Journal of Production Research*, 2025.
- [19] J. Yin et al. Rethinking supply chain planning: A generative paradigm (SCPA). *arXiv:2509.03811*, 2025.
- [20] S. AlMahri, L. Xu, and A. Brintrup. Automating supply chain disruption monitoring via an agentic AI approach. *arXiv:2601.09680*, 2026.
- [21] Y. Zhang et al. Multi-agent RAG for supply chain knowledge bases. In *KDD 2025 Workshop*, 2025.
- [22] Systematic analysis of generative AI for supply chain transformation. *ScienceDirect*, 2025.
- [23] L. Zheng, W.-L. Chiang, Y. Sheng, et al. Judging LLM-as-a-judge with MT-bench and chatbot arena. In *Proceedings of NeurIPS*, 2023.
- [24] J. Gu et al. A survey on LLM-as-a-judge. *arXiv:2411.15594*, 2024.
- [25] H. Li et al. LLMs-as-judges: A comprehensive survey on LLM-based evaluation methods. *arXiv:2412.05579*, 2024.
- [26] S. Shankar et al. Who validates the validators? Aligning LLM-assisted evaluation with human preferences. *arXiv:2404.12272*, 2024.
- [27] E. Evans. *Domain-Driven Design: Tackling Complexity in the Heart of Software*. Addison-Wesley, 2003.

A. MCP Tool Reference

The 35 MCP tools are listed below. Naming convention: *system_verb_noun*. All tools are read-only.

Tool	Description
SRM — Procurement	
srn_get_supplier	Supplier profile.
srn_list_suppliers	All suppliers.
srn_get_supplier_product	One supplier-product entry (cost, product link).
srn_list_supplier_products_by_supplier	All items offered by one supplier.
srn_list_supplier_products_by_product	All suppliers that sell a given product.
srn_get_supplier_order	One supplier order by ID.
srn_list_all_supplier_orders	All supplier orders.
srn_list_supplier_orders_by_supplier	Supplier orders placed with a given supplier.
srn_list_supplier_orders_by_status	Supplier orders filtered by status.
srn_list_overdue_supplier_orders	Supplier orders past expected delivery.
srn_compare_suppliers	Side-by-side supplier performance.
WMS — Inventory	
wms_get_product	Product by ID.
wms_get_product_by_sku	Product by SKU.
wms_list_products	All products.
wms_check_inventory	Inventory status for one product.
wms_list_low_stock	Products below reorder point.
wms_get_stock_movements	Movement history for one product.
wms_list_all_inventory	Full inventory snapshot.
wms_get_warehouse	Warehouse by ID.
OMS — Sales	
oms_get_customer_order	Customer order with line items.
oms_get_customer	Customer by ID.
oms_list_customer_orders_by_customer	Customer orders for one customer.
oms_list_customer_orders_by_status	Customer orders filtered by status.
oms_list_overdue_customer_orders	Customer orders past required date.
oms_list_all_customer_orders	All customer orders.
oms_list_customers	All customers.
oms_search_customer_orders_by_product	Customer orders containing a product.
TMS — Shipping	
tms_get_shipment	Shipment with tracking events.
tms_track_customer_order	All shipments for a customer order.
tms_list_exception_shipments	Shipments in exception state.
tms_get_carrier	Carrier by ID.
tms_list_shipments_by_status	Shipments filtered by status.
tms_list_carriers	All carriers.

B. Scenario Metadata

Table B.1 lists all 27 scenarios with their expected system coverage, minimum tool count, and human-baseline estimate. These metadata fields are used by the aggregator to compute cross-system coverage, tool efficiency, and time-savings metrics.

Table B.1: Scenario metadata.

ID	Level	UC	Expected systems	Min tools	Human min
L1-01	L1	UC-4	WMS	1	2
L1-02	L1	UC-4	OMS	1	2
L1-03	L1	UC-3	SRM	1	2
L1-04	L1	UC-4	TMS	1	2
L2-01	L2	UC-1	OMS	1	5
L2-02	L2	UC-4	OMS	1	6
L2-03	L2	UC-4	OMS	1	5
L2-04	L2	UC-4	WMS	1	5
L2-05	L2	UC-3	SRM	1	6
L2-06	L2	UC-4	TMS	1	5
L3-01	L3	UC-1	OMS, TMS	2	10
L3-02	L3	UC-4	WMS, OMS	2	12
L3-03	L3	UC-3	SRM	1	15
L3-04	L3	UC-3	SRM	1	10
L3-05	L3	UC-1	OMS, TMS	2	8
L3-06	L3	UC-4	WMS	1	5
L4-01	L4	UC-2	OMS, TMS, WMS, SRM	4	25
L4-02	L4	UC-2	TMS, OMS, WMS	3	20
L4-03	L4	UC-3	SRM	2	15
L4-04	L4	UC-4	OMS, TMS, WMS, SRM	4	25
L4-05	L4	UC-2	WMS, OMS	3	20
L4-06	L4	UC-1	OMS, TMS, WMS	3	18
L5-01	L5	UC-4	SRM, WMS, OMS	6	40
L5-02	L5	UC-4	OMS, TMS, WMS, SRM	6	45
L5-03	L5	UC-5	OMS, WMS, SRM, TMS	6	35
L5-04	L5	UC-3	SRM	4	30
L5-05	L5	UC-5	WMS, OMS, SRM	5	35

Glossary

ReAct Reasoning and Acting; an agent paradigm that interleaves a reasoning step (thought) and an action step (tool call) in a single loop [3](#)

Acronyms

AI Artificial Intelligence [1](#), [4](#)

LLM Large Language Model [1–4](#), [10](#), [19](#), [20](#)

MCP Model Context Protocol [1–3](#), [8–10](#), [19](#), [20](#), [24](#)

OMS Order Management System [iv](#), [1](#), [5](#), [12](#), [13](#)

PO Purchase Order [6](#), [9](#), [13](#)

SRM Supplier Relationship Management [iv](#), [1](#), [5](#), [13](#)

TMS Transport Management System [iv](#), [1](#), [5](#), [12](#), [13](#)

WMS Warehouse Management System [iv](#), [1](#), [5](#), [12](#), [13](#)